



Honeypot & Honeynet as a Service

Technische Dokumentation

Lehrveranstaltung: Softwarearchitektur · Semester 4

Autor: Stefan, Alex, Alex · Fachhochschule · 2025

1 Einleitung

Die vorliegende Dokumentation beschreibt die Konzeption und Implementierung eines Honeytrap-Systems im Rahmen der Lehrveranstaltung Softwarearchitektur. Ziel des Projekts ist es, das Verhalten von Cyber-Angeifern zu simulieren, zu erfassen und auszuwerten, ohne dabei echte Systeme oder Daten zu gefährden.

Ein Honeytrap ist ein absichtlich verwundbares Schein-System, das Angreifer anlocken soll. Es enthält keine echten Daten, zeichnet aber jede Aktion des Angreifers detailliert auf. Auf diese Weise lassen sich Angriffsmuster, verwendete Tools und Taktiken studieren, ohne reale Infrastruktur zu gefährden.

Lernziele dieses Projekts

Verstehen von Angriffstechniken (SQL Injection, XSS, Path Traversal, Brute Force) · Implementierung eines HTTP-Servers mit Python Stdlib · Strukturiertes Logging und Log-Analyse · Rechtliche und ethische Aspekte von Honeytraps

1.1 Honeytrap vs. Honeytrap

Der Begriff Honeytrap bezeichnet ein einzelnes Schein-System, das einen Server, eine API oder eine Datenbank imitiert. Ein Honeytrap hingegen ist ein Netzwerk aus mehreren Honeytraps, das eine gesamte Infrastruktur simuliert und komplexere Angriffsanalysen ermöglicht. Dieses Projekt implementiert einen Honeytrap.

Merkmal	Honeytrap	Honeytrap
Umfang	Ein einzelnes System	Netzwerk aus mehreren Systemen
Komplexität	Gering bis mittel	Hoch
Datenmenge	Begrenzt	Umfangreich
Einsatzbereich	Kleinere Umgebungen	Enterprise / Forschung
Analyse	Einzelne Angriffe	Komplexe Angriffsketten

2 Systemarchitektur

Das System besteht aus drei Python-Skripten, die unabhängig voneinander gestartet werden können. Der Honeypot-Server läuft dauerhaft und nimmt Verbindungen entgegen, während der Angriffs-Simulator und der Log-Analyzer nur bei Bedarf ausgeführt werden.

2.1 Komponenten-Übersicht

Datei	Rolle	Abhängigkeiten
honeypot.py	HTTP-Server, Kern des Systems	Python Stdlib (http.server, json, re, threading)
attacker_sim.py	Angriffs-Simulator für Tests	Python Stdlib (urllib.request)
log_analyzer.py	Log-Auswertung und Threat Report	Python Stdlib (json, collections)
honeypot.log	JSON-Lines Log-Datei (auto-erstellt)	—

2.2 Datenfluss

Der typische Ablauf bei einem eingehenden Angriff:

1. Angreifer sendet HTTP-Request an Port 8080
2. HoneypotHandler.do_GET() / do_POST() wird aufgerufen
3. detect_attacks() scannt Path und Body auf Angriffsmuster
4. AttackTracker.record() speichert den Eintrag thread-sicher
5. Logger schreibt JSON-Eintrag in honeypot.log
6. Server antwortet mit Fake-Daten (Status 200 oder 401)
7. log_analyzer.py wertet honeypot.log bei Bedarf aus

2.3 Verfügbare Endpunkte

Methode	Pfad	Beschreibung	Fake-Inhalt
GET	/	App-Root	App-Name, Server-Version
GET	/api/customers	Kundendaten	3 Fake-Kundendatensätze (JSON)
GET	/api/admin	Admin-Credentials	Fake Username + Passwort-Hash
GET	/api/config	Systemkonfiguration	Fake DB-Host, DB-User, API-Key
GET	/admin	Adminbereich	Hinweis auf /admin/login
POST	/admin/login	Login-Formular	Immer 401 Unauthorized
GET	/stats	Echtzeit-Statistiken	Angriffs-Counter, Top-IPs

3 Implementierung

3.1 honeypot.py — Der Kern

Das Herzstück des Projekts ist ein HTTP-Server auf Basis von Pythons eingebautem `http.server`-Modul. Er benötigt keine externen Bibliotheken und läuft auf jeder Standard-Python-Installation.

3.1.1 Fake-Daten (FAKE_DATA)

Das Dictionary `FAKE_DATA` enthält überzeugend wirkende, aber vollständig erfundene Kundendaten, Admin-Credentials und Konfigurationswerte. Der absichtlich sichtbare Passwort-Hash und die Datenbank-Credentials dienen als Köder — ein echter Angreifer würde diese sofort als wertvoll einschätzen und versuchen, sie zu nutzen.

3.1.2 Attack-Detection (`detect_attacks`)

Die Methode `detect_attacks()` prüft jeden eingehenden Request mit fünf Regex-Patterns:

Angriffs-Typ	Erkannte Muster	Beispiel
sql_injection	UNION, SELECT, OR 1=1, --	?id=1 OR 1=1--
xss	<script>, javascript:, onerror=	<script>alert(1)</script>
path_traversal	../, %2e%2e	/../../etc/passwd
command_injection	;, , \$(), && + Befehle	; cat /etc/shadow
login_bruteforce	/login, /auth, /admin	POST /admin/login

3.1.3 AttackTracker

Der AttackTracker hält alle Request-Einträge thread-sicher im Arbeitsspeicher (kein externes Redis oder ähnliches nötig). Er verwendet `threading.Lock()` für atomare Operationen und `defaultdict` für das automatische Anlegen neuer IP-Einträge. Die `get_stats()`-Methode liefert aggregierte Kennzahlen für den `/stats`-Endpunkt.

3.1.4 Authentizitätssignale

Um den Honeypot glaubwürdig zu machen, werden folgende Täuschungsmaßnahmen eingesetzt:

- Server-Header gibt eine veraltete Apache-Version zurück (Apache/2.2.14 Ubuntu)
- Login-Endpunkt hat eine künstliche Verzögerung (0.5 Sekunden) wie ein echtes System
- Fake-Daten enthalten realistische Struktur (IDs, E-Mail-Adressen, Kontostand)
- Endpunkte wie `/.env` und `/.git/config` liefern 404 (statt Fehler) — erwartetes Verhalten

3.2 attacker_sim.py — Der Simulator

Der Simulator führt einen realistischen Angriffs-Workflow in 7 Phasen durch. Er dient ausschließlich zum Testen des Honeypots in der Entwicklungsumgebung.

Phase	Technik	Ziel
1 — Reconnaissance	GET auf bekannte Pfade	Struktur der Anwendung erkunden
2 — SQL Injection	URL-kodierte Payloads	Datenbankzugriff umgehen
3 — XSS	Script-Tags im Query-Parameter	Code-Injection testen
4 — Path Traversal	Encoded ../ Sequenzen	Systemdateien lesen
5 — Brute Force	Wiederholte POST-Login-Requests	Admin-Zugang erzwingen
6 — Data Harvesting	GET auf /api/admin, /api/config	Credentials und Keys stehlen
7 — Stats abrufen	GET /stats	Logging-Ergebnis prüfen

3.3 log_analyzer.py — Die Auswertung

Der Log-Analyzer liest honeypot.log zeilenweise ein und parst jede Zeile als JSON. Anschließend aggregiert er die Daten mit collections.Counter und defaultdict und gibt einen formatierten Threat Report auf der Konsole aus.

Ausgabe-Sektionen des Reports:

- Überblick: Unique IPs, Gesamtzahl Requests, Erkannte Angriffe, HTTP-Methoden
- Angriffs-Typen: Häufigkeit mit ASCII-Balkendiagramm
- Top-IPs: Meist-angreifende IP-Adressen
- Meist angefragte Pfade: Beliebteste Ziel-Endpunkte
- Letzte 5 Angriffs-Einträge: Timestamp, IP, Pfad, Angriffs-Typ

4 Sicherheit, Recht & Ethik

4.1 Rechtslage in Österreich und der EU

Honeypots sind in Österreich und der gesamten EU grundsätzlich legal, sofern sie passiv beobachten. Die relevanten Rechtsgrundlagen sind:

- Strafgesetzbuch §118a: Betrieb eines Honeypots ist erlaubt — passive Beobachtung ist keine Straftat
- DSGVO Art. 5 & 6: IP-Adressen sind personenbezogene Daten; Logs unterliegen der Datenschutzgrundverordnung
- Aktives Hack-Back (Gegenangriffe) ist auch bei Angreifern illegal
- Entrapment: Angreifer dürfen nicht aktiv zu Angriffen verleitet werden

4.2 Technische Sicherheitsmaßnahmen

Pflicht: Isolierte Umgebung

Dieses Projekt darf ausschließlich in einer isolierten Testumgebung betrieben werden. Niemals auf einem öffentlich erreichbaren Server ohne Netzwerk-Segmentierung deployen.

Empfohlene Schutzmaßnahmen für einen produktiven Einsatz:

- Server nur auf 127.0.0.1 binden (localhost) oder dediziertes Honeypot-VLAN verwenden
- Log-Dateien regelmäßig rotieren (z.B. mit logrotate unter Linux)
- Monitoring auf ungewöhnlich hohe Request-Raten einrichten
- Incident-Response-Plan bereithalten falls ein Angreifer weiteren Schaden anrichtet
- Docker-Container für zusätzliche Isolation (nächste Ausbaustufe)

5 Demo-Ergebnisse

Die folgende Tabelle zeigt die Ergebnisse eines vollständigen Testdurchlaufs mit `attacker_sim.py` gegen eine lokal laufende Instanz von `honeypot.py`.

Metrik	Wert
Gesamt-Requests	11
Erkannte Angriffe	7
Attack-Rate	64 %
Erkannte Typen	5 (SQL Injection, XSS, Path Traversal, Brute Force, Credential Harvesting)
Unique IPs	1 (127.0.0.1 — Localhost-Test)
Laufzeit	ca. 3 Sekunden (inkl. künstlicher Login-Verzögerungen)

5.1 Angriffs-Verteilung

Angriffs-Typ	Erkannt	Anteil
SQL Injection	3	43 %
Brute Force	3	43 %
Path Traversal	2	29 %
XSS	1	14 %
Cmd Injection	0	0 %

5.2 Beispiel-Logeintrag

Ein typischer SQL-Injection-Eintrag in `honeypot.log` (formatiert):

```
{
  "timestamp": "2025-06-08T10:23:45Z",
  "ip": "127.0.0.1",
  "method": "GET",
  "path": "/api/customers?id=1%20OR%201=1--",
  "user_agent": "python-attacker-sim/1.0",
  "body_snippet": null,
  "attack_types": ["sql_injection"]
}
```

6 Mögliche Erweiterungen

Das vorliegende Projekt stellt eine Minimal-Implementierung dar. Für einen produktionsreifen Einsatz oder eine vertiefte Forschungsarbeit bieten sich folgende Erweiterungen an:

6.1 Kurzfristig (wenige Stunden Aufwand)

- Email-Alerts: Benachrichtigung bei kritischen Angriffs-Typen via smtp lib
- Geo-IP Mapping: Herkunftsland der Angreifer via ip-api.com REST-API
- Docker-Deployment: Dockerfile für isolierte Ausführung
- Passwort-Logging verbessern: Brute-Force-Wörterbücher analysieren

6.2 Mittelfristig

- Grafana + InfluxDB: Echtzeit-Dashboard mit Zeitreihen-Visualisierung
- GenAI-Fake-Daten: GPT/Claude generiert überzeugende Fake-Einträge dynamisch
- HTTPS-Unterstützung: TLS-Zertifikat für realistischere HTTPS-Anwendungen
- Honeynet-Ausbau: Mehrere Honeypots (DB, Mail, FTP) in einem Docker-Netzwerk

6.3 Langfristig / Forschung

- Automatische Threat-Intelligence: Abgleich mit CVE-Datenbanken
- Machine Learning: Klassifizierung neuer Angriffsmuster
- Sharing mit Community: Upload zu abuse.ch oder ähnlichen Plattformen

7 Quellen & Referenzen

Nr.	Quelle
[1]	Viresh Garg, CISSP — Honeypot and Honeynet as a Service. Medium, August 2024.
[2]	NIST SP 800-150 — Guide to Cyber Threat Information Sharing. NIST, 2016.
[3]	OWASP — Testing for SQL Injection (WSTG-INPV-05). owasp.org, 2023.
[4]	Python Software Foundation — http.server — HTTP servers. docs.python.org.
[5]	Python Software Foundation — threading — Thread-based parallelism. docs.python.org.
[6]	Bundesamt für Sicherheit in der Informationstechnik (BSI) — Honeypots und Honeynets.
[7]	RFC 7235 — Hypertext Transfer Protocol (HTTP/1.1): Authentication. IETF, 2014.

Anhang A — Quickstart

Alle Schritte zum Starten des Projekts auf einem neuen System:

A.1 Voraussetzungen

- Python 3.10 oder neuer
- Keine externen Packages erforderlich (nur Python Standard Library)
- Windows, macOS oder Linux

A.2 Ausführung

Terminal 1 — Honeypot starten:

```
cd honeypot
python honeypot.py
```

Terminal 2 — Angriffe simulieren (während Terminal 1 läuft):

```
python attacker_sim.py
```

Terminal 3 — Log auswerten:

```
python log_analyzer.py
```

Browser — Echtzeit-Statistiken:

```
http://127.0.0.1:8080/stats
```

A.3 Erwartete Ausgabe (Terminal 2)

Nach dem Durchlauf von `attacker_sim.py` sollte die Konsole folgendes zeigen:

```
[1] Reconnaissance
    GET /                → 200 OK
    GET /api/customers → 200
[2] SQL Injection
    OR 1=1                → 200 (erkannt im Log)
    UNION SELECT          → 200 (erkannt im Log)
...
[7] Honeypot-Statistiken
    Gesamt Requests:      11
    Erkannte Angriffe:    7
```